

2. Estimation de paramètres et histogrammes

I. Probabilités et histogramme

I.1. Génération de nombres aléatoires

Pour générer des nombres de manière aléatoire avec Python, on utilise le module **random**. Comme pour le module **numpy**, avant de l'utiliser, il faut l'importer :

```
import random
```

On peut alors utiliser différents générateurs aléatoires, on citera par exemple **random.randint(a,b)** qui renvoie de manière uniforme un entier entre a et b (inclus) ou **random.random()** qui renvoie de manière uniforme un réel entre 0 et 1.

Exercice d'application 1. Définir :

- 1) Une fonction **alea** qui renvoie de manière uniforme un réel entre 0 et 10.
- 2) Une fonction **liste** prenant en paramètre des entiers a, b et n et qui renvoie une liste avec n tirages aléatoires indépendants d'entiers choisis de manière uniforme entre a et b .
- 3) Une fonction **point** qui renvoie un point de $[-1, 1] \times [-1, 1]$ où l'abscisse et l'ordonnée sont indépendantes et tirées de manière uniforme.

On peut simuler de nombreuses lois, je vous renvoie à l'aide de Python pour plus d'informations.

I.2. Histogramme

Il est parfois utile pour voir comment se répartit un jeu de données de tracer un histogramme pour voir la répartition des données (par exemple combien d'éléments sont compris entre 0 et 1, combien entre 1 et 2, etc.). Pour construire et tracer des histogrammes, nous aurons besoin de certaines fonctions des modules **matplotlib** et **numpy**. On importe ce dont on a besoin en début de script. Voici un exemple complet permettant de tracer un histogramme donnant la répartition de 100 tirages aléatoires de nombres entre 0 et 10 :

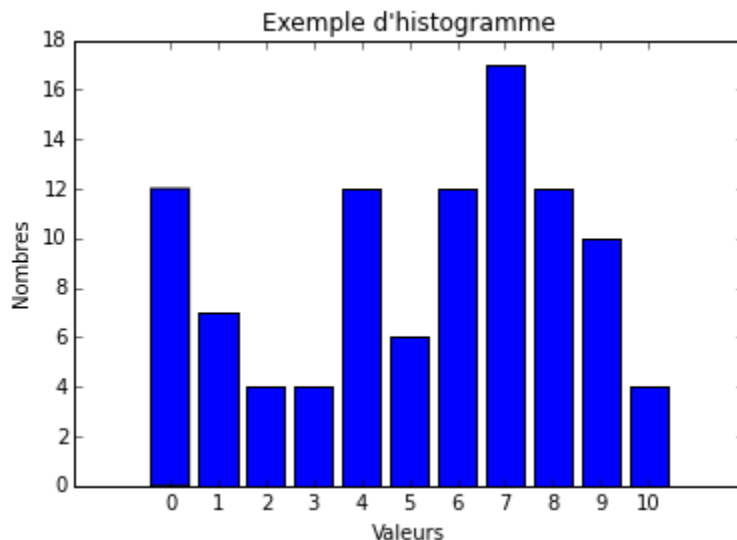
```
import random
import matplotlib.pyplot as plt
import numpy as np

# liste de 100 valeurs comprises entre 0 et 10
valeurs=[]
for k in range(100):
    valeurs=valeurs+[random.randint(0,10)]
print(valeurs) # Affichage des valeurs

inter = [-0.5, 0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5, 10.5]
plt.hist(valeurs, bins=inter, rwidth=0.8) # Création de l'histogramme
plt.xlabel('Valeurs')
plt.xticks(np.arange(0, 11))
plt.ylabel('Nombres')
plt.title("Exemple d'histogramme")
plt.show()
```

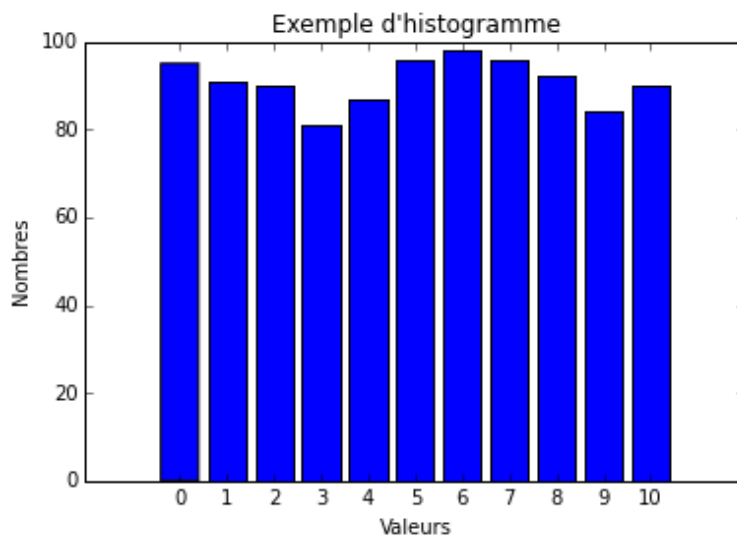
La fonction définissant l'histogramme **plt.hist** prend donc différentes choses en paramètres :

- Le jeu de donnée (ici la liste **valeurs**).
- Le paramètre **bins** qui permet de définir les différents intervalles (ici on regardera donc le nombre d'éléments dans $[-0.5, 0.5[$, puis $[0.5, 1.5[$, $[1.5, 2.5[$, etc.).
- Le paramètre **rwidth** est optionnel, il n'est ici que pour avoir des barres qui ne se touchent pas (rendu plus joli). On peut bien entendu l'enlever.

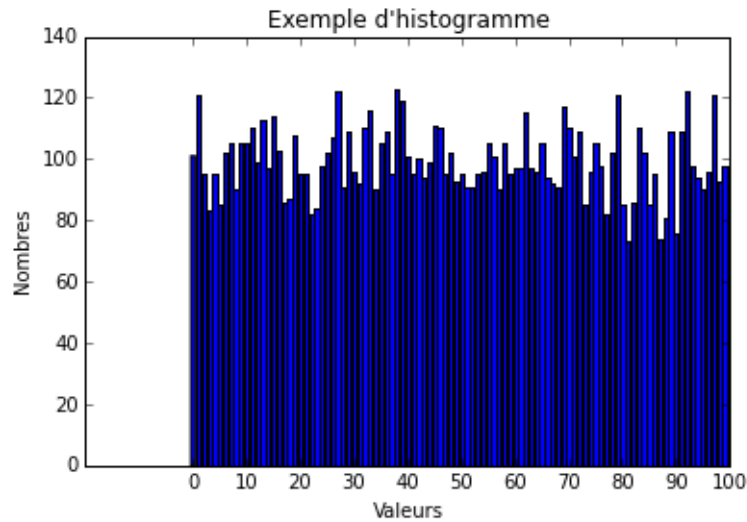


Il existe d'autres manières de définir les intervalles de répartition de l'histogramme, je vous renvoie à l'aide de Python pour plus d'informations, ces paramètres là permettant déjà beaucoup de souplesse.

Exercice d'application 2. Recommencer quelques fois le tracé précédent pour 1000 valeurs entre 0 et 10. Qu'observe-t-on ?

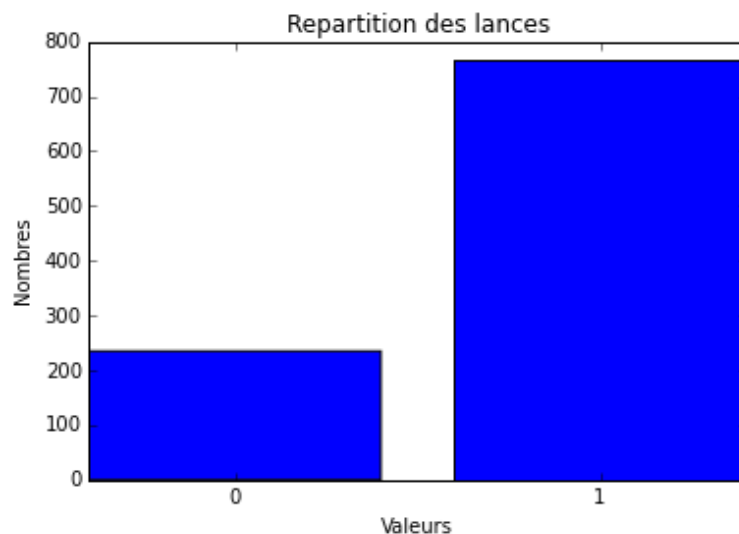


Exercice d'application 3. Recommencer mais cette fois pour 10000 valeurs entre 0 et 100 en redéfinissant bien entendu **inter** pour qu'il prenne en compte les 101 valeurs possibles.



Exercice d'application 4. On cherche à voir si on tire 1000 points au hasard dans $[-1, 1] \times [-1, 1]$ combien tombe en moyenne dans le disque de centre O et de rayon 1.

- 1) Écrire une fonction **test** prenant en paramètre $(x, y) \in [-1, 1] \times [-1, 1]$ et renvoyant 1 si (x, y) est dans le disque (autrement dit si $x^2 + y^2 \leq 1$) et 0 sinon.
- 2) Tracer l'histogramme permettant de visualiser combien de points sont tombés dans le disque et combien de points sont tombés en dehors. Vous devriez avoir un tracé qui ressemble à ceci :



- 3) Vérifier graphiquement qu'on a environ entre 7 et 8 chances sur 10 de tomber dans le disque avec ce procédé.

II. Estimation

Bien entendu, estimer un paramètre de manière visuelle n'est pas très précis. On va voir dans cette partie comment non seulement estimer une valeur, mais également avoir une idée de la précision de l'approximation.

II.1. Estimation d'un paramètre

L'idée ici est d'utiliser la loi des grands nombres (que vous verrez en détail l'an prochain en mathématiques). L'idée étant que si $(X_k)_{k \in \mathbb{N}}$ sont des variables aléatoires de même espérance $\mathbb{E}(X)$, alors,

sous certaines hypothèses (que l'on considère vérifiées en pratique), alors $\lim_{n \rightarrow +\infty} \frac{1}{n} \sum_{k=1}^n X_k = \mathbb{E}(X)$.

Pour estimer une valeur moyenne, il suffit donc de simuler pour n assez grand n simulations et de calculer la valeur moyenne des résultats.

Exercice d'application 5. En reprenant l'exercice d'application 4, estimer pour $n = 100$ la probabilité pour un point de tomber dans le disque. Recommencer avec $n = 1000$ et $n = 10000$.

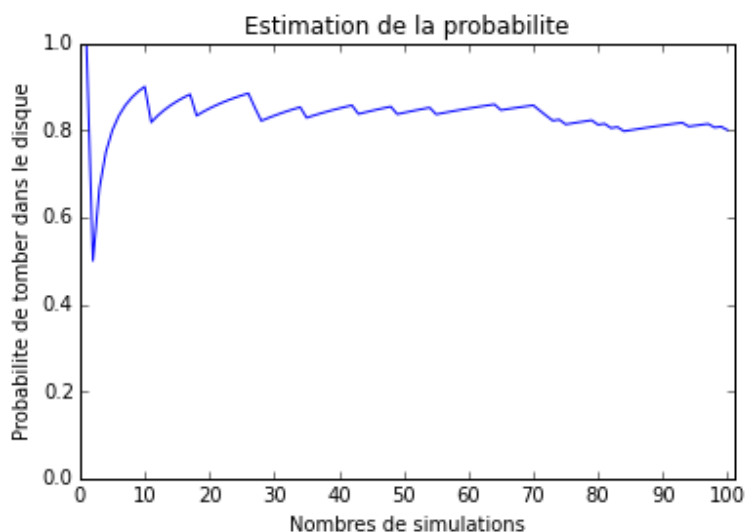
II.2. Visualisation de l'estimation

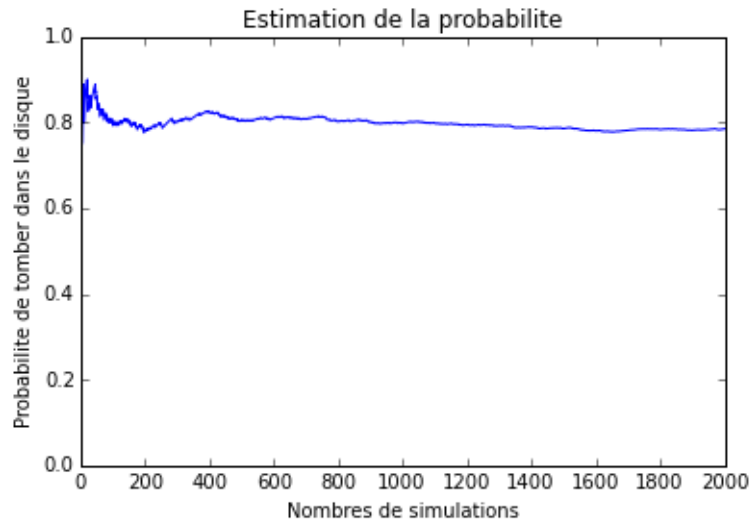
La méthode précédente, même si elle donne un résultat, possède deux défauts :

- On ne sait pas quelle valeur de n prendre pour avoir une « bonne » approximation (autrement dit pour avoir peu de variations sur la valeur « réelle »).
- On ne connaît pas la précision de l'approximation, même si elle semble « bonne ».

Dans cette partie, nous allons régler le premier problème à l'aide d'un tracé de fonction. L'idée est de tracer en fonction de k variant entre 1 et n la suite des valeurs $\frac{\sum_{j=1}^k X_j}{k}$ (autrement dit la suite des valeurs moyennes estimées). Quand la courbe ne « bouge » plus beaucoup, on peut estimer que la valeur de n à ce niveau est suffisante pour avoir une bonne approximation. Attention, il est important ici de ne réaliser que n simulations en tout (on réutilise donc à chaque fois les anciens résultats) et non pas de re-simuler à chaque étape k expériences aléatoires, sinon on observera beaucoup de bruit...

Exercice d'application 6. Mettre en oeuvre cette méthode pour déterminer à partir de quelle valeur de n on a l'impression de bien estimer la probabilité de tomber dans le disque avec l'expérience précédente et vérifier que cette probabilité est proche de $\frac{\pi}{4} \approx 0,785$.





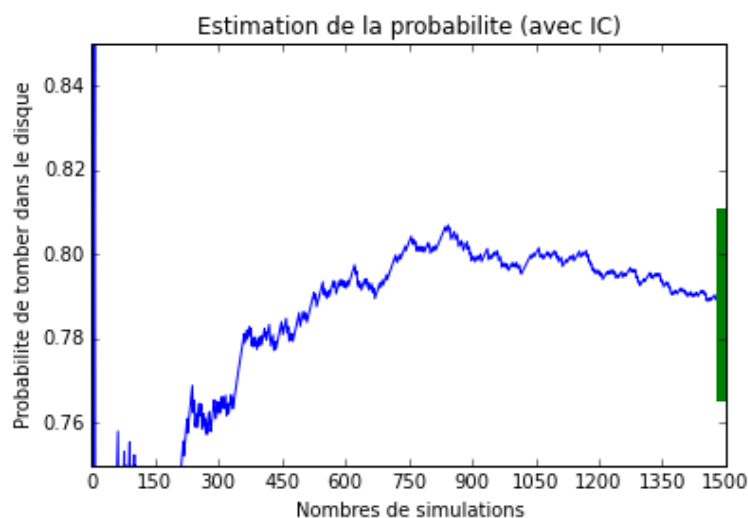
II.3. Intervalle de confiance (pour aller plus loin)

Bien entendu, même si la courbe ne bouge pas beaucoup, ce n'est pas très rigoureux pour justifier que l'approximation est bonne. On utilise pour cela la notion d'intervalle de confiance qui permet de générer un intervalle aléatoire qui à chaque génération a une certaine probabilité de contenir le paramètre estimé. Ainsi, la génération d'un intervalle de confiance à 95% permettra d'affirmer que si le paramètre estimé n'est pas dans l'intervalle, c'est qu'il n'y avait que 5% de chance de générer un tel intervalle.

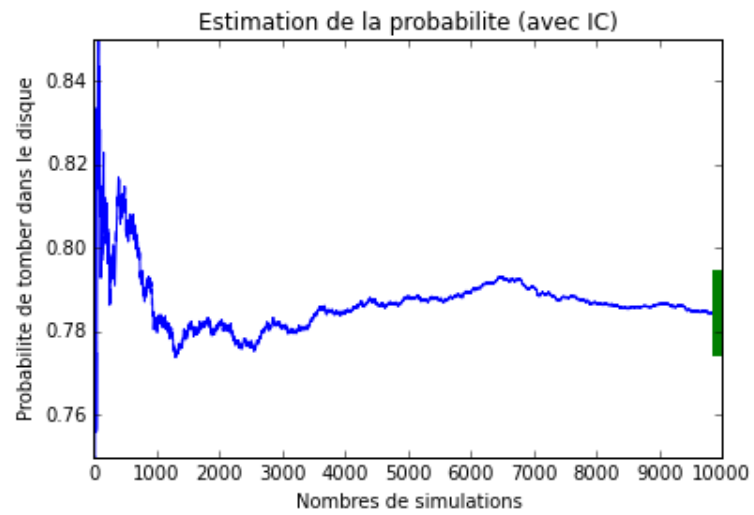
En pratique, pour le générer, on utilise le théorème central limite (hors programme en 1A et en 2A). On procède en pratique de la manière suivante :

- On calcule le centre de l'intervalle de la manière précédente en calculant $m_n = \frac{1}{n} \sum_{k=1}^n X_k$.
- On calcule $V_n = \frac{1}{n-1} \sum_{k=1}^n (X_k - m_n)^2$ (c'est un estimateur de la variance). *Il faut donc bien avoir gardé les X_k en mémoire pour le calculer.*
- L'intervalle de confiance à 95% est alors de la forme $\left[m_n - 1,96\sqrt{\frac{V_n}{n}}, m_n + 1,96\sqrt{\frac{V_n}{n}} \right]$.
- On trace alors sur le tracé précédent l'intervalle de confiance à l'étape n .

Exercice d'application 7. Toujours sur l'exemple précédent, représenter l'intervalle de confiance à 95% à l'étape $n = 1500$.



Bien entendu, plus n augmente, plus l'intervalle est petit (et donc l'estimation est précise). Voici un tracé pour $n = 10000$:



III. Correction des exercices

Exercice d'application 1.

a)

```
def alea():  
    return(10*random.random())
```

b)

```
def liste(a,b,n):  
    L=[]  
    for k in range(n):  
        L=L+[random.randint(a,b)]  
    return(L)
```

c)

```
def point():  
    x=2*random.random()-1  
    y=2*random.random()-1  
    return([x,y])
```

Exercice d'application 2. On reprend le code de l'énoncé mais en allant jusqu'à 1000.

```
valeurs=[]  
for k in range(1000):  
    valeurs=valeurs+[random.randint(0,10)]  
print(valeurs) # Affichage des valeurs  
  
inter = [-0.5, 0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5, 10.5]  
plt.hist(valeurs, bins=inter, rwidth=0.8) # Création de l'histogramme  
plt.xlabel('Valeurs')  
plt.xticks(np.arange(0, 11))  
plt.ylabel('Nombres')  
plt.title("Exemple d'histogramme")  
plt.show()
```

On observe que le tirage a bien l'air uniforme.

Exercice d'application 3.

```
valeurs=[]  
for k in range(10000):  
    valeurs=valeurs+[random.randint(0,100)]  
print(valeurs) # Affichage des valeurs  
  
inter=[-0.5+k for k in range(0,101)]  
  
plt.hist(valeurs, bins=inter, rwidth=0.8) # Création de l'histogramme  
plt.xlabel('Valeurs')  
plt.xticks(np.arange(0, 101,10))  
plt.ylabel('Nombres')  
plt.title("Exemple d'histogramme")  
plt.show()
```

Le tirage a encore bien l'air uniforme.

Exercice d'application 4.

1)

```
def test(x,y):
    if x**2+y**2<=1:
        return(1)
    else:
        return(0)
```

2) On réutilise bien sûr les fonctions test et point :

```
valeurs=[]
for k in range(1000):
    coord=point()
    valeurs=valeurs+[test(coord[0],coord[1])]
print(valeurs) # Affichage des valeurs

inter=[-0.5,0.5,1.5]
#inter = [-0.5, 0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5, 10.5]
plt.hist(valeurs, bins=inter, rwidth=0.8) # Création de l'histogramme
plt.xlabel('Valeurs')
plt.xticks(np.arange(0, 2))
plt.ylabel('Nombres')
plt.title("Repartition des lances")
plt.show()
```

3) En utilisant la fonction **sum**, on peut compter le nombre de 1 dans la liste **valeurs**. Sur le tracé du polycopié, on en a 765 sur 1000 lancés. On a donc bien entre 7 et 8 chances sur 10 de tomber dans le disque à chaque tirage.

Exercice d'application 5. On calcule donc à chaque fois le nombre de points qui tombent dans le disque divisé par le nombre de tirages. Si on note p_n la probabilité estimé en fonction du nombre de tirages, on obtient $p_{100} = 0,76$, $p_{1000} = 0,781$ et $p_{10000} = 0,7816$.

```
def simulation(n):
    compteur=0.
    for k in range(n):
        coord=point()
        compteur=compteur+test(coord[0],coord[1])
    return(compteur/n)
```

Exercice d'application 6. Voici le code pour à la fois la simulation et le tracé pour 100 lancés :

```
def simulationliste(n):
    compteur=0.
    L=[]
    for k in range(n):
        coord=point()
        compteur=compteur+test(coord[0],coord[1])
        L=L+[compteur/(k+1)]
    return(L)
```



```

abscisse=[k for k in range(1,101)]
plt.plot(abscisse,simulationliste(100))
plt.xlabel('Nombres de simulations')
plt.xticks(np.arange(0, 101,10))
plt.axis([0,101,0,1])
plt.ylabel('Probabilite de tomber dans le disque')
plt.title("Estimation de la probabilite")
plt.show()

```

Pour 1000 lancés, il suffit de faire aller les abscisses de 1 à 1001, d'évaluer la fonction simulationliste à 1000 et pour la légende de faire aller l'axe des abscisses de 1 à 1000 tous les 100 par exemple. On a ici l'impression d'une stabilisation vers 1500 simulations environ...

Exercice d'application 7. On effectue tout dans la même fonction et on superpose les tracés.

```

def intervalledeconfiance(n):
    compteur=0.
    liste=[]
    L=[]
    for k in range(n):
        coord=point()
        resultat=test(coord[0],coord[1])
        compteur=compteur+resultat
        liste=liste+[resultat]
        L=L+[compteur/(k+1)]
    moyenne=compteur/n
    variance=0.
    for j in range(n):
        variance=variance+(liste[j]-moyenne)**2
    variance=variance/(n-1)
    return(L, [moyenne-1.96*np.sqrt(variance/n), moyenne+1.96*np.sqrt(variance/n)])

simulation=intervalledeconfiance(1500)
trace=simulation[0]
IC=simulation[1]

abscisse=[k for k in range(1,1501)]
plt.plot(abscisse,trace)
plt.plot([1500,1500],[IC[0],IC[1]],color='green',linewidth='10')
plt.xlabel('Nombres de simulations')
plt.xticks(np.arange(0, 1501,150))
plt.axis([0,1501,0.75,0.85])
plt.ylabel('Probabilite de tomber dans le disque')
plt.title("Estimation de la probabilite")
plt.show()

```